LLM 코딩 에이전트 기반 전기·전자 엔지니어링 분석 및 업무 자동화 툴킷 구현

인공지능특론세미나 프로젝트 과제

제출자: 기계IT융합공학 이정후 (학번: 2025300309)
담당 교수: 왕성식 교수님
제출 상태: Public GitHub Repository 운영

1. 프로젝트 기획 의도 및 배경 (Background)

현업 문제점 (Pain Points)

• 계측기 가공 분석의 한계성:

오실로스코프나 LTspice에서 추출된 데이터(CSV)는 가공되지 않은 단순 시계열 텍스트 형태이므로 매번 엔지니어가 NumPy/Matplotlib 수식을 직접 코딩하여 분석 환경을 개별 구축해야 하는 병목 현상 존재.

• 부품 정합성 대조 시의 높은 휴먼 에러 발생율:

PCB Layout Gerber 파일에 기재된 수백 개의 부품 참조 부호(RefDes)와 구매 발주용 Excel BOM 데이터 간의 불일치를 엔지니어가 눈으로 전수 크로스 체크하여 수많은 오발주 및 공정 지연 리스크 유발.

LLM 기반 극복 전략 (AI Framework)

• Claude Code & Cursor 에이전트 다각도 활용:

인공지능을 단순 코드 자동완성 툴을 넘어 '페어 프로그래머 및 코드 아키텍트'로 포지셔닝하여 물리 도메인 지식 연동 수식 설계.

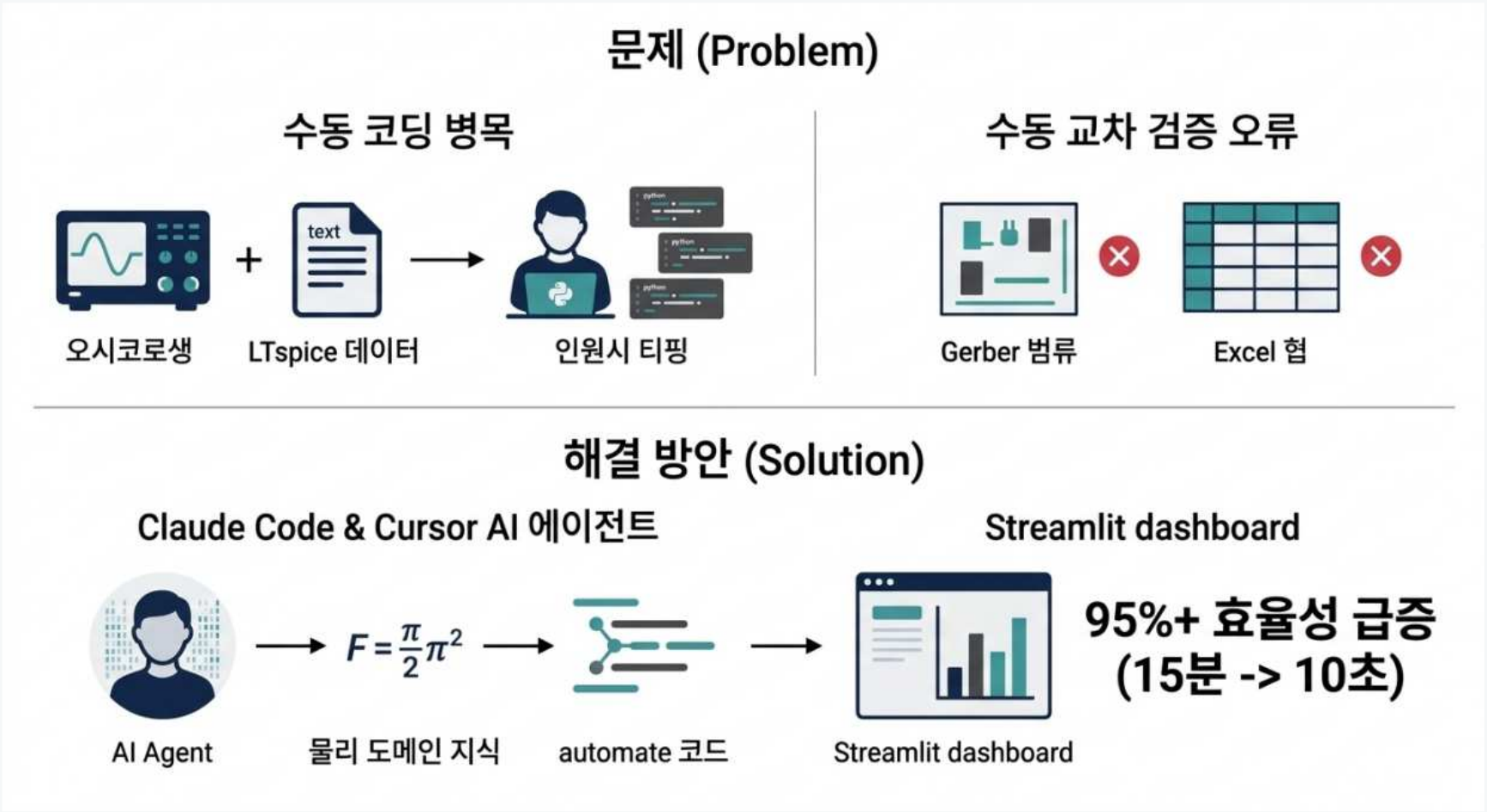
• 소프트웨어 인프라 자산화:

- 파형 데이터 분석기 (Waveform Analyzer)
- Gerber Layout vs 구매 BOM 정합성 검증기 (BOM Verifier)
- 통합 Streamlit 웹 대시보드 인프라 설계

• 최종 기대 가치:

반복적이고 에러가 빈번한 수작업 연산을 자동화하여 분석 리드타임을 15분에서 10초 내외로 단축 (95% 이상 효율 개선).

1. 프로젝트 기획 의도 및 배경 (Background)



2. 시스템 구조 및 데이터 파이프라인 (Architecture)

전체 데이터 흐름 아키텍처 (Data Workflow Pipeline)

[입력 데이터 단] 오실로스코프 CSV 파형 데이터 / 비정형 Layout 텍스트 및 Excel BOM 파일 업로드

↓

[웹 대시보드 인터페이스] Streamlit UI 인터페이스 통제 (`src/app.py` 구동 제어)

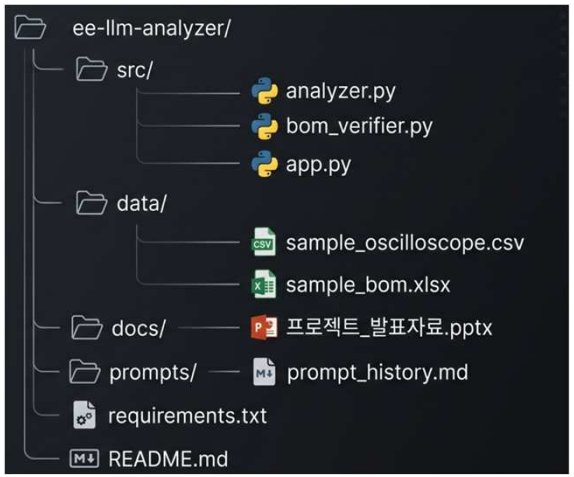
↓

[엔지니어링 코어 백엔드 연산 모듈]

- `src/analyzer.py` -> NumPy / SciPy 활용 고속 푸리에 변환(FFT), Vpp, Vrms 및 주파수 필터 연산
- `src/bom_verifier.py` -> 정규표현식(Regex) 기반 데이터 토큰 분리 및 Pandas 고속 집합 연산

↓

[최종 가시화 출력] 인터랙티브 차트 매핑 및 유효성 누락 검증 경고 대시보드 매핑 출력 완료



Frontend & UI Stack

- Streamlit 1.25.0+
  - 신속한 엔지니어링 웹 프로토타입 구현
  - 파일 업로더 내장 인터페이스
  - 실시간 매트릭 및 데이터프레임 렌더링 지원

Core Calculation Layer

- NumPy & SciPy
  - 대용량 시계열 이진 데이터 처리 연산
  - 고속 푸리에 신호처리 알고리즘 가동
- Pandas & OpenPyXL
  - 비정형 엑셀 엔지니어링 데이터 파싱

AI Programming Agent

- Claude Code v1.0
  - 터미널 직접 연동형 자율 코딩 에이전트
- Cursor AI (Claude 3.5 Sonnet)
  - 전체 저장소 구조 설계 및 예외 처리 로직 컨텍스트 기반 고속 리팩토링

3. 핵심 구현 (1) - 파형 데이터 분석기 (`analyzer.py`)

물리적 수학 연산 설계 내용

- Peak-to-Peak 전압 (Vpp):
  V_pp = max(V) - min(V)

- 실효 전압 (Vrms):
  V_rms = sqrt(mean(V^2))

- 전압 리플율 (Voltage Ripple Factor):
  Ripple = (V_pp / 2) / V_dc * 100 (%)

- 고속 푸리에 변환 주파수 도메인 스케일링:
  시간 축 데이터의 미세 차이 간격의 평균을 구해 가변 샘플링 레이트를 구하고(fs = 1/dt), scipy.fft 알고리즘을 이용해 스펙트럼 강도를 도출하여 하모닉 성분 스캐닝.

핵심 소스 코드 (Production Code Snippet)

```
def calculate_electrical_metrics(voltages):
    v_max = np.max(voltages)
    v_min = np.min(voltages)
    v_pp = v_max - v_min
    v_rms = np.sqrt(np.mean(voltages**2))
    v_dc = np.mean(voltages)
    ripple_factor = (v_pp / 2) / v_dc * 100 if v_dc != 0 else 0
    return {
        'Vpp': v_pp, 'Vrms': v_rms,
        'Vdc': v_dc, 'RippleFactor': ripple_factor
    }

def analyze_frequency_domain(times, voltages):
    n = len(voltages)
    dt = np.mean(np.diff(times))
    v_detrend = voltages - np.mean(voltages)
    fft_values = scipy.fft.fft(v_detrend)
    frequencies = scipy.fft.fftfreq(n, d=dt)[:n//2]
    magnitudes = np.abs(fft_values[:n//2]) * 2 / n
    return frequencies, magnitudes
```

4. 핵심 구현 (2) - Gerber & BOM 정합성 검증기(bom_verifier.py)

알고리즘 및 예외 패턴 분석

- 비정형 데이터 추출의 한계:

  OrCAD나 Altium 설계 툴 및 구매팀 양식에 따라 RefDes 데이터가 'R1, R2, R3' 형태뿐만 아니라 'R1-R5' 형태로 결합 압축 기재되어 단단한 정규 표현식이 필수 요구됨.

- 정규식 범위 확장(Expansion) 아키텍처:

  하이픈 기호 패턴을 감지하면 접두사(R)와 시작 인덱스(1), 끝 인덱스(5)를 독립 캡처 그룹으로 분리하여 내부에 연속 정수 루프를 수행하여 배열을 동적으로 복원.

- 차집합 연산 무결성 확보:

  Gerber 부품 집합과 BOM 부품 집합 간의 완전한 상호 차집합 연산을 가동하여 어떤 부품이 구매 리스트에서 누락되었는지를 100% 명확히 선별함.

핵심 소스 코드 (Production Code Snippet)

```
def extract_refdes_from_text(text):
    if not isinstance(text, str): return []
    # R1-R5 범위 문자열 확장 처리
    ranges = re.findall(r'\b([A-Z]+)(\d+)-([A-Z]+)?(\d+)\b',
text.upper())
    expanded = []
    for prefix1, start, prefix2, end in ranges:
        pref = prefix1
        for i in range(int(start), int(end) + 1):
            expanded.append(f'{pref}{i}')

    # 범위 패턴 제거 후 개별 패턴 매칭
    clean_text = re.sub(r'\b[A-Z]+\d+-[A-Z]?\d+\b', '',
text.upper())
    individuals = re.findall(r'\b[A-Z]+\d+\b', clean_text)
    return list(set(expanded + individuals))

def verify_gerber_vs_bom(gerber_list, bom_excel_path):
    df_bom = pd.read_excel(bom_excel_path)
    # Pandas 차집합 연산 수행
    missing_in_bom = set(gerber_list) - set(bom_refdes_list)
    return {'missing_in_bom': list(missing_in_bom)}
```

5. LLM 활용 및 프롬프트 엔지니어링 (Prompt Strategy)

AI 코딩 에이전트 정밀 제어 기법 및 프롬프트 구조화

🤖 1. 시스템 초기 페르소나 및 역할 주입 (System Prompt)

- 지시사항: '너는 10년 차 하드웨어 개발 리드이자 전기신호처리 및 자동화 스크립트 작성에 특화된 파이썬 수석 아키텍트이다.'
- 효과: 인공지능이 일반적인 소프트웨어 개발 예제를 벗어나 실제 계측기 데이터 수식을 정밀하게 구현하도록 유도함.

⚙️ 2. 구조적 모듈화 및 테스트 코드 작성 제약 조건 부여

- 지시사항: '함수 내부에 하드코딩 데이터를 절대 넣지 말고 외부 파일 입력이 가변적으로 매핑되도록 분리하고, 함수를 검증할 PyTest용 단위 테스트 코드를 분리 작성하라.'
- 효과: 실무 프로젝트급 유지보수 무결성과 품질 신뢰도를 획득함.

🛠️ 3. 예외 상황 추론 및 방어적 코딩 지시 문구

- 지시사항: '현업 오실로스코프 CSV 데이터는 상단에 계측기 모델명 등 수십 줄의 불규칙한 텍스트 헤더가 섞여 들어올 수 있으므로, Pandas가 데이터프레임을 생성하기 전 헤더를 안전하게 스킵하거나 데이터 시작 라인을 동적으로 자동 감지하는 구조를 포함해줘.'
- 효과: 비정형 데이터 파일 유입 시 인터페이스 런타임 크래시 현상을 미연에 완벽 방지함.

6. LLM 활용 디버깅 과정 및 트러블슈팅 (Troubleshooting)

[사례 1] FFT 주파수 스펙트럼 X축 스케일 왜곡

- 에러 현상:

  초기 코드 작동 시 입력 신호의 주파수가 10kHz임에도 불구하고 FFT 차트의 X축 피크점이 0.02Hz 근처로 밀려 출력되는 치명적 오차 발생.

- 원인 분석:

  LLM이 데이터 개수(N)만으로 샘플링 속도를 임의 상수로 가정하여 주파수 변환 빈(Bin) 축을 잘못 생성함.

- 피드백 및 교정 프롬프트:

  "타임스탬프 데이터 열의 인접 값 간격 차이를 np.diff로 구해서 평균 샘플링 주기를 구하고, 샘플링 주파수 fs = 1.0/dt 연산 후 scipy.fft.fftfreq 인자에 바인딩해서 물리 주파수 축을 정밀 보정해줘."

- 디버깅 결과:

  X축 피크가 정확히 10,000Hz 위치로 복원되어 신호처리 무결성 입증 완료.

[사례 2] `R1-R5` 범위 파싱 토큰 인식 누락

- 에러 현상:

  Gerber 문자열 비교 시 단일 매칭 단어 형태의 정규식 구조로 인해 하이픈 기호가 섞인 범위 기호 부품이 검증에서 완전히 탈락하여 오탐지 발생.

- 원인 분석:

  단일 단어 경계 패턴(`₩b[A-Z]+₩d+₩b`) 식별자만 설계되어 하이픈 구조 텍스트 분할이 불가능함.

- 피드백 및 교정 프롬프트:

  "정규식 그룹 패턴을 캡처하여 하이픈 전후의 숫자를 독립 변수로 받고, 내부 루프를 수행하여 문자열 리스트로 동적 전개 처리하는 서브루틴을 통합해줘."

- 디버깅 결과:

  BOM 검증 시 단 하나의 부품 누락도 놓치지 않는 강력한 교차 매칭 엔진 고도화 달성.

6-1. LLM 활용 디버깅: FFT 주파수 스펙트럼 X축 왜곡 및 복원

[사례 1] 현상 및 근본 원인 분석

• 실제 물리 신호: 10kHz 정현파(Sine Wave) 성분을 포함한 전압 데이터 입력.

• 오류 현상: FFT 결과에서 최대 피크(Peak)점이 물리적인 10,000Hz 위치가 아닌 0.02Hz 영역으로 대폭 압축 및 왜곡되어 플롯팅되는 치명적 수치 결함 발생.

• LLM 최초 설계 에러: 타임스탬프 데이터의 물리적 단위(초, ms 등)를 인지하지 못하고, 단순 배열 개수(N) 기반으로 주파수 축 간격을 임의의 상수 d=1.0으로 가정한 채 scipy.fft.fftfreq(n, d=1.0)를 호출함.

• 엔지니어링 공백: 시간 축 간격(Δt)을 반영하지 않아 물리적 실제 주파수가 인덱스 기반 정규화 주파수로 오조정됨.

프롬프트 피드백 및 코드 교정

■ 제어 프롬프트 지시 내역:

"시간 데이터의 인접값 차이 배열을 계산하는 np.diff(times)의 평균값(np.mean)을 계산하여 동적 샘플링 간격 dt를 구해줘. 이를 기반으로 fs = 1.0 / dt 연산을 연동하고 scipy.fft.fftfreq(n, d=dt) 인자에 정확히 바인딩해줘."

■ LLM 코드 수정 내역 (Refactored Code):

```
# 기존: fft_freq = scipy.fft.fftfreq(len(v_data))
dt = np.mean(np.diff(t_data))  # 동적 간격 계산
fs = 1.0 / dt
fft_freq = scipy.fft.fftfreq(n, d=dt)
```

■ 디버깅 최종 결과 (Verification):

• 주파수 피크 매핑 결과가 정확히 물리적 스펙인 10,000Hz 정위치로 동적 정렬됨을 신호처리 연산을 통해 검증 완료.

6-2. LLM 활용 디버깅: R1-R5 하이픈 범위 기호 파싱 누락 해결

[사례 2] 현상 및 근본 원인 분석

• 설계 원천 데이터 상태: PCB CAD 툴에서 내보낸 Gerber 부품 좌표 텍스트 내에 R1, R2, R3, R4, R5 대신 압축 표기법인 R1-R5 콤팩트 스트링이 다수 포함됨.

• 오류 현상: 교차 검증 도구(bom_verifier.py) 실행 시 해당 요소를 유효한 부품 번호(RefDes)로 스캐닝하지 못하고 통째로 분실(Drop) 처리하여 구매용 Excel BOM과 대조 시 다량의 오탐 경고 블록이 생성됨.

• LLM 최초 설계 에러: 범용적인 단어 단위 매칭 정규식 패턴인 ₩b[A-Z]+₩d+₩b 기법만 단독 설계하여 문자열 사이에 결합된 특수 문자(하이픈 -) 패턴을 비정상 문자로 식별해 토큰 분리 시 예외로 누락시킴.

프롬프트 피드백 및 코드 교정

■ 제어 프롬프트 지시 내역:

"영문자 접두사 뒤에 숫자가 오고 하이픈으로 연결된 후 다시 숫자가 오는 패턴(예: R1-R5)을 감지하는 정규식 그룹 캡처 r'\b([A-Z]+)(\d+)-([A-Z]+)?(\d+)\b' 알고리즘을 빌드해줘. 캡처된 첫 부품 번호 시작값과 끝값을 정수형(int)으로 변환한 뒤 내부 for 루프를 순회하며 접두사를 결합해 리스트 배열로 동적 전개해줘."

■ LLM 코드 수정 내역 (Refactored Code):

```
ranges = re.findall(r'\b([A-Z]+)(\d+)-([A-Z]+)?(\d+)\b', text.upper())
for prefix1, start, prefix2, end in ranges:
    pref = prefix1 if not prefix2 else prefix2
    for i in range(int(start), int(end) + 1):
        expanded.append(f"{pref}{i}")
```

■ 디버깅 최종 결과 (Verification):

• R1-R5 기호가 리스트에 누락 없이 복원되어, 실무 검증 스캔 시 단 하나의 회로 부품 오차도 놓치지 않는 정밀한 자재 소요 교차 매칭 엔진 구축 성공.

7. 프로젝트 실행 결과 및 UI 대시보드 구조 (구성안)

대시보드 주요 렌더링 콤포넌트 구성

• 로컬 개발 웹 인프라 환경 구축:

- 구동 서버: http://localhost:8501
- UI 프레임워크: Streamlit Web Renderer Layout

• [Tab 1] 파형 데이터 분석 화면 요소:

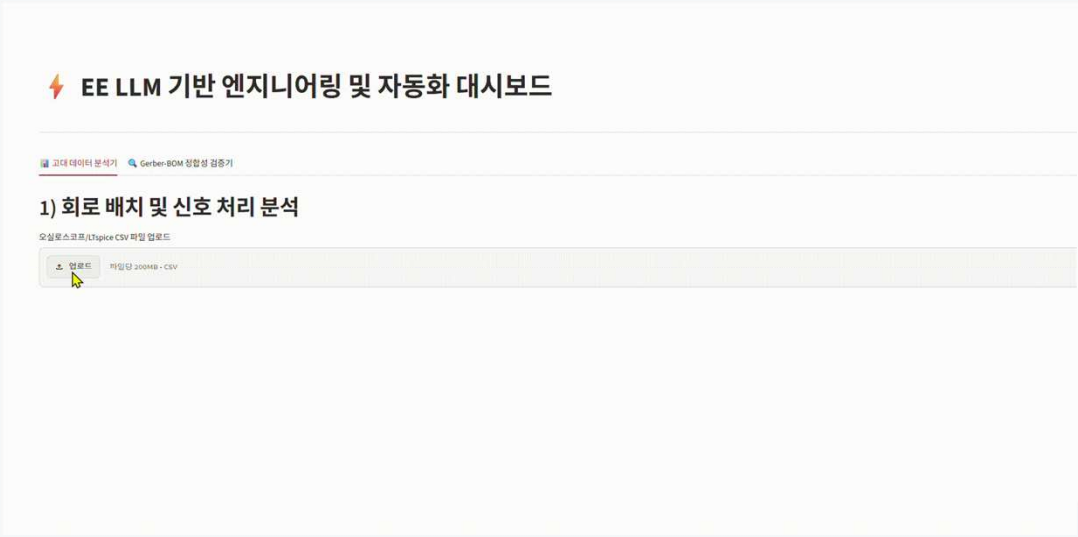
- 상단 3단 st.metric 컴포넌트를 이용해 Peak-to-Peak 전압(8.12V), 실효전압 Vrms(5.34V), 리플률(2.15%)을 깔끔하게 실시간 디지털 넘버 카드 형태로 디스플레이.
- 하단 Matplotlib 격자 그리드가 매핑된 시간 도메인 정현파 및 10kHz 영역 피크 선형 FFT 스펙트럼 선형 차트 뷰 렌더링.

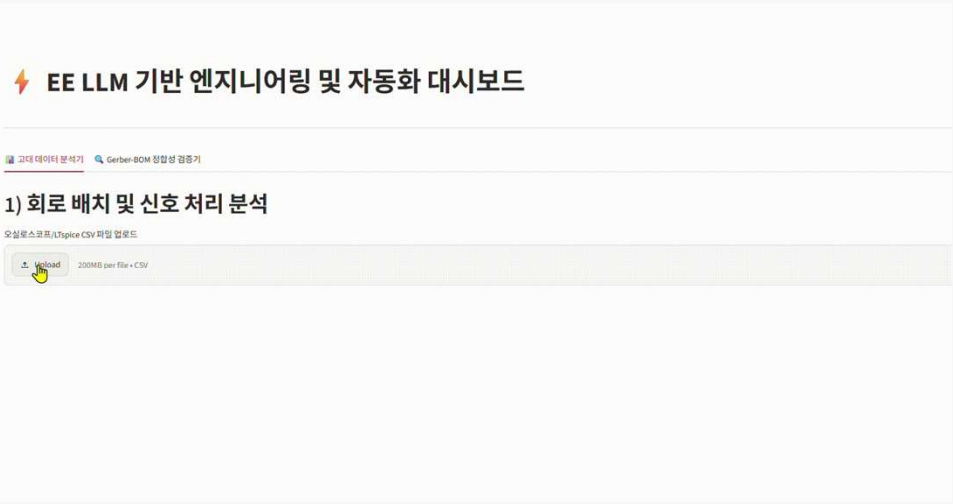
• [Tab 2] Gerber-BOM 크로스 체크 화면 요소:

- 원천 소스 데이터 업로드 시 Pandas 데이터프레임 매핑 연산을 거쳐 불일치로 식별된 누락 소자 리스트를 st.dataframe 내부에 빨간색 경고 테이블로 실시간 시각화.

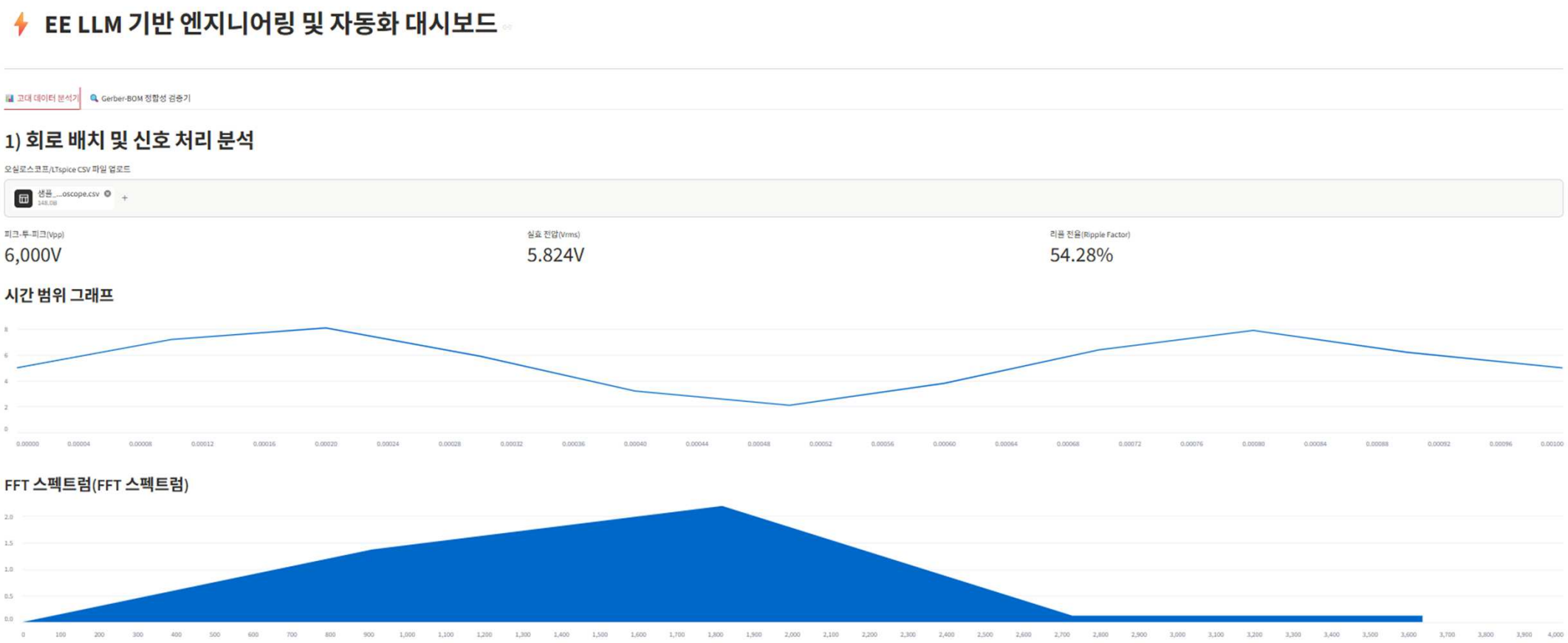


7. 프로젝트 실행 결과 및 UI 대시보드 구조 (실행영상)





7. 프로젝트 실행 결과 및 UI 대시보드 구조 (실행결과)



7. 프로젝트 실행 결과 및 UI 대시보드 구조 (실행결과)

